

PROJECT HANDOVER DOCUMENT

Pune Metro LRP Team Portal

Livelihood Restoration Plan

Prepared: 27 July 2026

Repository: github.com/mitconbengaluru-create/LRP-PUNE

Branch: main (production) | development (active work)

Table of Contents

1. Executive Summary.....	3
2. Project Purpose & Context.....	3
3. Technology Stack	4
4. Repository & Code Structure	5
5. Database Architecture	7
6. Backend API — Full Reference	8
7. Frontend Application	10
8. Authentication & RBAC System	11
9. Master Workbook System (Core Engine).....	12
10. Document Hub & AI Pipeline	13
11. Storage System	14
12. Admin Modules.....	15
13. Environment Variables Reference	16
14. Local Development Setup	17
15. Docker / Production Deployment.....	18
16. Testing.....	18
17. Known Issues, Technical Debt & Gotchas	19
18. Key Files Quick Reference	20

1. Executive Summary

The Pune Metro LRP Team Portal is a full-stack internal web application built for the Livelihood Restoration Plan (LRP) team managing the Pune Metro Rail project's impact on displaced households. It serves two primary purposes:

Public Reporting Portal

A polished, data-driven website showing six pillars of livelihood restoration progress to stakeholders, with live metrics pulled from the database and visualised through interactive charts, 3D animations, and animated counters.

Internal Team Management System

A password-protected staff portal for data entry operators, administrators, and supervisors to:

- Import and manage the master household survey Excel workbook
- Search and edit individual family data records
- Upload PDF/DOCX/image progress reports and have AI (Gemini) extract metrics automatically
- Review extracted metrics and commit them to the live database
- Generate updated Excel workbooks from database records
- Manage users, audit logs, backups, and system notifications

2. Project Purpose & Context

The Six LRP Pillars

The Pune Metro LRP organises livelihood restoration for displaced households into six measurable pillars:

#	Pillar	Description
1	Institution Building & Community Development	CHS, SHG, Anganwadi formation
2	Accessibility & Urban Integration	Student/professional transit pass programs
3	Government Schemes Convergence	Welfare scheme linkage (health, food, income, ID)
4	Health & Well-being	Health camps, mobile vans, ABHA registration
5	Employment & Job Placement	Corporate partnerships, referrals, placements
6	Skill Building & Vocational Training	Training programs, certification, empowerment

Master Workbook

The project revolves around an Excel Master Workbook (currently Revision 29, based at a hardcoded path). This workbook stores all household (family) survey data. The system can:

- Import a workbook revision into PostgreSQL
- Edit individual family fields via the UI
- Generate new workbook revisions by writing DB edits back into Excel cells

3. Technology Stack

Backend

Category	Technology
Runtime	Node.js (v18+)
Framework	Express.js v5
ORM	Prisma v7
Database	PostgreSQL (Neon hosted)
Auth	JWT (HTTP-Only Cookies)
File Parsing	pdf-parse, mammoth, xlsx, officeparser
AI Extraction	Google Gemini 2.5 Flash (@google/genai)
Storage	Local Disk (default) / MinIO (optional)
Queue	BullMQ + ioredis (installed, partially wired)
Excel Engine	ExcelJS
Validation	Zod
Security	Helmet, CORS, bcryptjs, express-rate-limit
Logging	Morgan, custom JSON audit log
Testing	Jest + Supertest
Dev Tools	Nodemon, TypeScript (dev only)

Frontend

Category	Technology
Framework	React 19 (Vite 8)
Routing	React Router DOM v7
State	Zustand v5
Data Fetching	TanStack React Query v5
HTTP	Axios
UI Animations	Framer Motion
3D Graphics	Three.js / @react-three/fiber + drei
Charts	Recharts
Styling	TailwindCSS v4 (via Vite plugin)
Forms	React Hook Form + Zod
Icons	Lucide React + Google Material Symbols

4. Repository & Code Structure

Top-level layout of the monorepo (*marks the most important files to know):

```

lrp-pune_main/
├── backend/                                # Express API server
│   ├── prisma/
│   │   ├── schema.prisma                  # * Full DB entity definitions
│   │   └── migrations/                    # Auto-generated Prisma migration files
│   └── src/
│       ├── admin/                         # Admin-specific controllers
│       │   ├── audit/                     # auditController.js, auditLogger.js
│       │   ├── backup/                     # backupController.js
│       │   ├── dashboard/                  # dashboardController.js (workbook stats)
│       │   ├── notifications/              # notificationController.js
│       │   ├── users/                      # userController.js (CRUD admin users)
│       │   └── workbooks/                  # workbookController.js (import/activate)
│       ├── config/
│       │   ├── db.js                       # * Prisma client singleton
│       │   ├── minio.js                     # MinIO client + bucket init
│       │   ├── permissions.js               # * RBAC permissions matrix
│       │   └── rateLimit.js                 # Express rate limiter config
│       ├── controllers/
│       │   ├── authController.js           # login / logout / register
│       │   └── documentController.js       # Document hub endpoints
│       ├── master-workbook/                # * Workbook engine (core feature)
│       │   ├── comparison/                 # comparisonService.js
│       │   ├── controllers/                 # workbookController.js
│       │   ├── export/                     # exportController.js, exportRoutes.js
│       │   ├── family/                     # familyController.js, familyRoutes.js
│       │   ├── generator/                  # generatorService.js, monthlyExpansion.js
│       │   ├── history/                    # historyController.js, historyRoutes.js
│       │   ├── importer/                   # dbImporter.js (Excel → PostgreSQL)
│       │   ├── parser/                     # workbookParser.js
│       │   ├── repositories/                # workbookRepository.js
│       │   ├── routes/                     # workbookRoutes.js
│       │   ├── search/                     # searchController.js, searchRoutes.js
│       │   ├── services/                   # workbookService.js
│       │   ├── updates/                     # updateController.js, updateRoutes.js
│       │   └── validation/                  # validationController.js, validationEngine.js,
│       │                                           # validationRoutes.js, validationRunner.js
│       ├── middleware/
│       │   ├── authenticate.js              # * JWT cookie verification
│       │   ├── authorize.js                 # * Permission gate
│       │   └── validates.js                 # Zod schema validator middleware
│       ├── routes/
│       │   ├── adminRoutes.js               # * /admin & /api admin routes
│       │   ├── authRoutes.js               # /auth/login, /auth/logout, /auth/me
│       │   ├── dashboardRoutes.js           # /api/dashboard
│       │   ├── documentRoutes.js            # /api/documents
│       │   ├── errorHandler.js             # Global Express error handler
│       │   └── pillarRoutes.js              # /api/pillars/:id
│       ├── scripts/
│       │   └── create_default_admin.js      # Seed script for initial admin
│       └── services/
│           ├── aiService.js                 # * Gemini AI extraction engine
│           ├── auditService.js              # logAudit() helper
│           ├── authService.js               # login, logout, register, validateUser
│           ├── dashboardService.js          # Public homepage pillar data aggregator
│           └── documentService.js           # * Full ingestion pipeline

```

```

├── pdfServices.js      # Legacy PDF parser wrapper
├── pillarService.js    # Pillar-specific data fetcher
├── storage/
│   ├── index.js       # Storage factory (local vs MinIO)
│   ├── localStorageService.js
│   ├── minioStorageService.js
│   └── storageService.js # * Unified storage facade
├── storage/
│   └── uploadConfig.js # Multer configuration for documents
├── tests/              # Jest test suites
├── utils/
│   └── jwtHelper.js    # generateAccessToken()
├── validators/
│   ├── authValidator.js # Zod schemas for auth
│   └── extractedDataValidator.js # Zod schema for AI output
├── .env                # * Secrets file (see §13)
├── server.js           # * Entry point
├── Dockerfile          # Backend container build
├── package.json
├── frontend/          # React 19 Vite SPA
│   └── src/
│       ├── admin/      # Admin panel components (Phase 5)
│       │   ├── AdminLayout.jsx # Admin sidebar layout shell
│       │   ├── audit/AuditCentre.jsx
│       │   ├── backup/BackupManager.jsx
│       │   ├── dashboard/AdminDashboard.jsx
│       │   ├── users/UserManagement.jsx
│       │   └── workbooks/WorkbookRepository.jsx
│       ├── components/
│       │   ├── NetworkErrorBoundary.jsx # Error boundary for network failures
│       │   ├── navigation/PageTraversal.jsx # Pillar prev/next navigation
│       │   └── public/                  # Navbar, AnimatedCounter, ThreePillarAnimation, etc.
│       ├── config/
│       │   └── navigationFlow.js # Pillar navigation sequence config
│       ├── hooks/
│       │   ├── useDashboard.js # React Query hook for dashboard data
│       │   └── usePillar.js    # React Query hook for pillar data
│       ├── lib/
│       │   └── rbac.js         # Frontend RBAC permission helpers
│       ├── master-data/      # Family data management UI
│       │   ├── components/   # DynamicForm, Field, Section
│       │   ├── editor/       # FamilyEditorPage, FieldEditor
│       │   ├── family/       # FamilyDetailPage
│       │   ├── history/      # HistoryViewer
│       │   ├── search/SearchPage.jsx # * Family search interface
│       │   ├── validation/   # ValidationHelper.js
│       │   └── workbook/     # WorkbookGenerationPage, WorkbookHistory,
│       │                       # ComparisonSummary, GenerationProgressDialog
│       ├── master-workbook/  # (Duplicate/legacy workbook components)
│       ├── pages/
│       │   ├── admin/
│       │   │   ├── AuditLogsPage.jsx
│       │   │   ├── DashboardPage.jsx # Main team dashboard
│       │   │   ├── DocumentsPage.jsx # * Document hub UI
│       │   │   └── LoginPage.jsx
│       │   └── pillars/
│       │       ├── PillarPage.jsx # Router/dispatcher for pillars 1-6
│       │       ├── PillarPage-1.jsx # Institution Building
│       │       ├── PillarPage-2.jsx # Accessibility
│       │       └── PillarPage-3.jsx # Govt. Convergence

```

```

├─── PillarPage-4.jsx # Health
├─── PillarPage-5.jsx # Employment
├─── PillarPage-6.jsx # Skill Building
├─── public/
│   ├─── HomePage.jsx # * Public landing page
│   └─── survey/
│       └─── SearchPage.jsx # Alternative survey search
├─── routes/
│   ├─── AppRoutes.jsx # * All route definitions
│   └─── Protectedroute.jsx # Auth guard component
├─── service/
│   ├─── adminService.js # Admin API calls
│   ├─── apiClient.js # * Axios instance + 401 interceptor
│   ├─── authService.js # login/logout/me API calls
│   ├─── dashboardService.js # Dashboard data API calls
│   ├─── documentService.js # Document hub API calls
│   └─── pillarService.js # Pillar data API calls
├─── store/
│   └─── authStore.js # * Zustand auth state (no localStorage)
├─── App.jsx # Root component
├─── index.css # Global TailwindCSS + custom tokens
├─── main.jsx # ReactDOM.createRoot entry
├── vite.config.js # Dev proxy + ngrok allowedHosts
├── Dockerfile
├── package.json
├── docker-compose.yml # Postgres + Redis + Backend + Frontend
├── README.md # AI agent onboarding guide
├── SOURCE_CODE_STRUCTURE.md # Legacy structure map
├── package.json # Root monorepo scripts

```

5. Database Architecture

Provider: PostgreSQL (hosted on Neon)

ORM: Prisma v7 with @prisma/adpater-pg

Schema file: backend/prisma/schema.prisma

Entity Relationship Overview

```
Admin —< LoginAudit
Family —< FamilyField >— WorkbookHeader
Family —< FieldUpdateHistory >— WorkbookHeader
WorkbookGeneration
Workbook
UploadLog
MetricAuditLog
PillarData
AuditLog
Backup
Notification
```

Table Definitions

Model	Purpose	Key Fields
Admin	Staff user accounts	id, email, username, passwordHash, role, isActive, lastLogin
LoginAudit	Every login attempt log	adminId, ipAddress, userAgent, loginTime, success
PillarData	Live metrics per pillar	pillar (0-6), key, value — unique on [pillar, key]
UploadLog	Document upload pipeline records	filename, storagePath, status (PROCESSING/COMPLETED/FAILED), extractedData, metricsUpdated
MetricAuditLog	Metric change history	pillar, metricKey, oldValue, newValue, updatedBy, source
Family	Household survey record	hhCode (immutable), ownerName (immutable search key), hhNumber, excelStartRow, excelEndRow
WorkbookHeader	Excel column definitions	excelColumn, headerName, section, dataType, isMonthly, isEditable
FamilyField	Individual cell values	familyId, headerId, excelRow, excelColumn, value
FieldUpdateHistory	All cell edits (full audit trail)	familyId, headerId, oldValue, newValue, updatedBy
WorkbookGeneration	Generated workbook records	revision, sourceWorkbook, generatedWorkbook, checksum, totalUpdatedCells
Workbook	All workbook versions (imported & generated)	name, revision, type (IMPORTED/GENERATED), status (ACTIVE/INACTIVE), filePath, checksum
Backup	Backup file records	filename, type (DATABASE/WORKBOOK), size, createdBy
Notification	System notifications	title, message, type (INFO/SUCCESS/WARNING/ERROR), read
AuditLog	General system audit trail	user, action, details, workbook, family

Important DB Constraints

- Family unique on [hhCode, ownerName]
- PillarData unique on [pillar, key]
- WorkbookGeneration unique on revision
- Workbook unique on revision
- FamilyField cascade-deletes when Family or WorkbookHeader is deleted
- Column A (hhCode) and Column E (ownerName) in the workbook are immutable by design

6. Backend API — Full Reference

Base URL (local): <http://localhost:5000>

Authentication: HTTP-Only cookie named token containing JWT

Auth Routes (/auth/*)

Method	Endpoint	Auth	Description
POST	/auth/login	None	Login with username/email + password. Rate-limited.
POST	/auth/logout	None	Clears auth cookie
GET	/auth/me	DONE JWT	Returns current logged-in user data
POST	/auth/register	None	Creates a new admin account (should be locked in prod)

Admin Routes (/api/* or /admin/*)

Method	Endpoint	Permission	Description
GET	/api/admin/dashboard	VIEW_DASHBOARD	Admin workbook & family system stats
GET	/api/workbooks	VIEW_DASHBOARD	List all workbook versions
GET	/api/workbooks/active	VIEW_DASHBOARD	Get currently active workbook
GET	/api/workbooks/:id	VIEW_DASHBOARD	Get specific workbook
POST	/api/workbooks/import	IMPORT_WORKBOOK	Upload & import an Excel workbook
PUT	/api/workbooks/active/:id	IMPORT_WORKBOOK	Switch active workbook
GET	/api/audit	VIEW_AUDIT_LOGS	Get system audit log entries
GET	/api/backups	VIEW_AUDIT_LOGS	List backups
POST	/api/backups/create	IMPORT_WORKBOOK	Create a new backup
GET	/api/backups/download/:id	VIEW_AUDIT_LOGS	Download backup file
GET	/api/admin/users	MANAGE_USERS	List all admin users
POST	/api/admin/users	MANAGE_USERS	Create new admin user
PUT	/api/admin/users/:id	MANAGE_USERS	Update user role/status
DELETE	/api/admin/users/:id	MANAGE_USERS	Delete admin user
GET	/api/notifications	VIEW_DASHBOARD	Get system notifications
POST	/api/notifications/read	VIEW_DASHBOARD	Mark notifications as read

Document Hub Routes (/api/documents/*)

Method	Endpoint	Auth	Description
POST	/api/documents/upload	DONE JWT	Upload document, trigger AI extraction pipeline
GET	/api/documents	DONE JWT	List all uploaded documents
GET	/api/documents/audit-logs	VIEW_AUDIT_LOGS	Get document-related audit logs
GET	/api/documents/:id	DONE JWT	Get single document details
GET	/api/documents/:id/download	DONE JWT	Download raw file
GET	/api/documents/:id/view	DONE JWT	View file inline (presigned URL or stream)

Method	Endpoint	Auth	Description
GET	/api/documents/:id/extraction	DONE JWT	Get AI-extracted JSON metrics
POST	/api/documents/:id/commit	DONE JWT	Commit selected metrics to live PillarData table
DELETE	/api/documents/:id	DONE JWT	Delete document record and backing file

Master Workbook Routes (/api/*)

Method	Endpoint	Description
GET	/api/search?q=<term>	Search families by name/hhCode/hhNumber
GET	/api/families/:id	Get family record with all fields
GET	/api/families/:id/history	Get all field update history for a family
PATCH	/api/families/:id/fields	Update family field values
POST	/api/validate	Validate a batch of field updates
GET	/api/validate/options	Get dropdown options for fields
GET	/api/history	Get global update history
GET	/api/workbook/generate	Generate new Excel revision from DB
GET	/api/workbook/compare	Compare two workbook revisions
GET	/api/workbook/history	Get list of generated workbook history
GET	/api/workbook/export/:id	Download a generated workbook

Public Routes

Method	Endpoint	Description
GET	/api/dashboard	Homepage pillar data, metrics, charts, stories
GET	/api/pillars/:id	Data for a specific pillar (1-6)
GET	/health	System health check (DB connectivity)

7. Frontend Application

Route Map

URL	Component	Auth	Description
/	HomePage	Public	Landing page with metrics, charts, stories
/pillars/:id	PillarPage → PillarPage-N	Public	Individual pillar detail (1–6)
/team/login	LoginPage	Public	Staff login
/team/dashboard	DashboardPage	DONE	Main team dashboard

URL	Component	Auth	Description
/team/documents	DocumentsPage	IMPORT_WORKBOOK	Document hub + AI extraction
/team/audit	AuditCentre	VIEW_AUDIT_LOGS	Audit log viewer
/team/search	MasterDataSearchPage	DONE	Search families by name/code
/team/survey	MasterDataSearchPage	DONE	Alias for /team/search
/team/survey/:id	FamilyEditorPage	DONE	Edit individual family record
/team/workbook	WorkbookGenerationPage	DONE	Generate workbook revision
/team/admin/dashboard	AdminDashboard	VIEW_DASHBOARD	Admin system stats
/team/admin/workbooks	WorkbookRepository	VIEW_DASHBOARD	Workbook version manager
/team/admin/audit	AuditCentre	VIEW_AUDIT_LOGS	Admin audit view
/team/admin/backup	BackupManager	VIEW_AUDIT_LOGS	Backup manager
/team/admin/users	UserManagement	MANAGE_USERS	User CRUD

API Client Design

- Base URL: window.location.origin (dynamic, works with ngrok)
- Credentials: withCredentials: true for HTTP-Only cookie support
- 401 Interceptor: Auto-clears auth state and redirects to /team/login?expired=true

State Management

- Zustand authStore: Holds user, role, isAuthenticated, isCheckingSession
- No localStorage/sessionStorage — all session state lives in HTTP-Only cookies
- TanStack React Query handles all server data caching and invalidation

8. Authentication & RBAC System

How Auth Works

- User submits credentials → POST /auth/login
- Backend validates password (bcrypt), generates JWT
- JWT stored in HTTP-Only cookie (not accessible to JS)
- React client calls GET /auth/me on app load to restore session
- On 401 responses, axios interceptor clears state and redirects to login

Role-Permission Matrix

Permission	SA1	SA2	SUPER_ADMIN	ADMIN1	ADMIN	ADMIN2	DATA_ENTRY_OP	READ_ONLY	DEVELOPER
IMPORT_WORKBOOK	DONE	DONE	DONE	DONE	DONE	✗	✗	✗	✗
EDIT_DATA	DONE	DONE	DONE	DONE	DONE	✗	DONE	✗	✗
GENERATE_WORKBOOK	DONE	DONE	DONE	DONE	DONE	✗	✗	✗	✗

Permission	SA1	SA2	SUPER_ADMIN	ADMIN1	ADMIN	ADMIN2	DATA_ENTRY_OP	READ_ONLY	DEVELOPER
DOWNLOAD_WORKBOOK	DONE	DONE	DONE	DONE	DONE	✗	DONE	DONE	✗
VIEW_AUDIT_LOGS	DONE	DONE	DONE	DONE	DONE	✗	✗	✗	DONE
MANAGE_USERS	DONE	DONE	DONE	✗	✗	✗	✗	✗	✗
VIEW_DASHBOARD	DONE	DONE	DONE	DONE	DONE	DONE	DONE	DONE	DONE

SA1 = SUPER_ADMIN1, SA2 = SUPER_ADMIN2. Column widths abbreviated for legibility.

Rate Limiting

Limiter	Development	Production
General	10,000 req / 15 min / IP	10 req / 15 min / IP
Login	1,000 req / 15 min / IP	10 req / 15 min / IP

⚠CRITICAL Production NODE_ENV must be set to "production" or rate limits will NOT be enforced.

9. Master Workbook System (Core Engine)

This is the most complex and critical part of the application.

Import Pipeline

- Admin uploads an Excel .xlsx file via /api/workbooks/import
- dbImporter.js clears existing FamilyField, Family, and WorkbookHeader tables
- Scans Excel headers (row 1 or multi-row merged headers)
- Detects firstDataRow by finding the first numeric HH Code in column A
- Inserts WorkbookHeader records for each column
- Scans each data row; creates a new Family record when a non-empty HH Code (col A) is found
- Inserts all cell values as FamilyField records
- Uses buffered writes (chunks of 40 families, 15,000 cells per Postgres insert) to handle WAN latency

Family Search

- Endpoint: GET /api/search?q=<term>
- Searches across ownerName, hhCode, and hhNumber using case-insensitive contains
- Also returns family members from column K

Family Data Editing

- Frontend fetches all WorkbookHeader records to build a dynamic form
- Immutable fields: columns A (hhCode) and E (ownerName) — isEditable = false
- Edits are saved to FamilyField and a FieldUpdateHistory record is created
- Pre-edit validation can be called via POST /api/validate

Workbook Generation

- Runs full database validation (validationRunner.js) — aborts if errors found
- Determines next revision number
- Loads the active workbook template from storage (or hardcoded fallback path)
- Expands monthly columns if needed (monthlyExpansion.js)
- Queries all FieldUpdateHistory and builds a deduplicated map of latest edits
- Applies edits cell-by-cell using ExcelJS (skips formula cells)
- Saves to temp file → calculates SHA-256 checksum → uploads via storageService
- Creates WorkbookGeneration and Workbook records in DB
- Creates a success Notification

⚠️CRITICAL KNOWN ISSUE: The template path is hardcoded in generatorService.js line 11.

```
const TEMPLATE_PATH = "c:/Users/--/Desktop/LRP-PUNE/Revision 29.xlsx";
```

This will break on any other machine or after deploying to Docker. It falls back to the activeWorkbook.filePath from the database if set.

10. Document Hub & AI Pipeline

Upload & Extraction Flow

```
User uploads file
↓
Multer stores file in memory (20MB max)
↓
storageService.upload() → saves to local disk or MinIO
↓
UploadLog created with status = "PROCESSING"
↓
File written to OS temp directory for parsing
↓
parseFile() reads content based on extension:
.pdf → pdf-parse (text) or base64 multimodal fallback
.docx → mammoth
.xlsx → XLSX + csv stringification
.pptx → officeparser
.txt / .csv → raw UTF-8
.png/.jpg/.jpeg → base64 multimodal
↓
aiService.extractMetrics() → Gemini 2.5 Flash
(fallback to Gemini 2.0 Flash on 503)
↓
Output validated against Zod schema
↓
Metrics stored as JSON in UploadLog.extractedData (status = "PENDING")
↓
UploadLog.status → "COMPLETED"
↓
User reviews metrics in UI → selects which to commit
↓
POST /api/documents/:id/commit
```

↓
 Committed metrics → upserted to PillarData
 Logged to MetricAuditLog
 Remaining pending → marked "EXCLUDED"

Supported File Types

Extension	Parser
.pdf	pdf-parse (text) → Gemini multimodal fallback
.docx	mammoth
.xlsx	xlsx (csv conversion)
.pptx	officeparser
.txt / .csv	UTF-8 read
.png, .jpg, .jpeg	Gemini multimodal (base64)

Gemini AI Integration

- Model: gemini-2.5-flash (primary), gemini-2.0-flash (fallback)
- API Key: GEMINI_API_KEY env var
- Retry Logic: 3 retries with exponential backoff (1s, 2s, 4s) for 503/429/500/504
- Output: Structured JSON array with schema [{pillar: int, key: string, value: string}]
- Pillar Schema: Detailed prompt in aiService.js defines exactly which metric keys to extract per pillar (0-6)

11. Storage System

The storage layer is pluggable — controlled by the STORAGE_PROVIDER environment variable.

Providers

Provider	STORAGE_PROVIDER	Description
Local Disk	local (default)	Saves files to backend/uploads/
MinIO	minio	S3-compatible object storage

Storage Interface (all providers implement)

- upload(file) → { storagePath: string }
- download(path) → { stream: ReadableStream }
- getViewUrl(path) → { type: "path" | "url", value: string }
- delete(path) → void

MinIO Config (when enabled)

Env Var	Default	Description
MINIO_ENDPOINT	localhost	MinIO host
MINIO_PORT	9000	MinIO port
MINIO_ACCESS_KEY	minioadmin	Access key
MINIO_SECRET_KEY	minioadmin	Secret key
MINIO_BUCKET	lrp-documents	Bucket name (auto-created)

12. Admin Modules

Admin Dashboard (/team/admin/dashboard)

Aggregates stats from PostgreSQL:

- Current active workbook
- Total families & members count
- Total rows and columns
- Pending changes since last generation
- Modified families count
- Generated workbooks count
- Storage usage estimate
- Recent audit activity

Workbook Repository (/team/admin/workbooks)

- Lists all imported and generated workbook versions
- Allows importing new Excel workbooks
- Allows switching the active workbook version

Audit Centre (/team/admin/audit / /team/audit)

Unified audit log from three sources:

- File-based JSON lines at backend/logs/audit.log (file operations)
- MetricAuditLog table (metric changes)
- LoginAudit table (authentication events)

Backup Manager (/team/admin/backup)

- Lists backup records from Backup table
- Allows creating new backups
- Allows downloading backup files

User Management (/team/admin/users)

- Lists all Admin records
- Create/update/delete users (requires MANAGE_USERS)

- Cannot delete the primary admin account
- Changes logged to AuditLog

13. Environment Variables Reference

Backend .env

Variable	Example Value	Required	Description
DATABASE_URL	postgresql://user:pass@host/db?sslmode=require	DONE	Neon/Postgres connection string
PORT	5000	DONE	Express server port
JWT_SECRET	<64-char hex>	DONE	JWT signing secret
NODE_ENV	development or production	DONE	Controls rate limits and error masking
GEMINI_API_KEY	xxxx	DONE	Google AI Studio API key
STORAGE_PROVIDER	local or minio	DONE	File storage backend
MINIO_ENDPOINT	localhost	MinIO only	MinIO server host
MINIO_PORT	9000	MinIO only	MinIO server port
MINIO_ACCESS_KEY	minioadmin	MinIO only	MinIO access key
MINIO_SECRET_KEY	minioadmin	MinIO only	MinIO secret key
MINIO_BUCKET	lrp-documents	MinIO only	Bucket name
REDIS_URL	redis://localhost:6379	Optional	BullMQ Redis URL
HTTPS	True	Optional	Hints for proxy SSL

Frontend (Optional)

Variable	Description
VITE_API_URL	Override backend URL (defaults to window.location.origin)

14. Local Development Setup

Prerequisites

- Node.js v18+
- A PostgreSQL instance (or Neon cloud account)
- A Gemini API Key from Google AI Studio

Step-by-Step

1. Clone the repository

```
git clone https://github.com/mitconbengaluru-create/LRP-PUNE.git
cd LRP-PUNE/lrp-pune_main
```

2. Backend Setup

```
cd backend
npm install
# Create backend/.env with all required variables (see §13)
# Then push the schema to your PostgreSQL database
npx prisma db push
# Seed the initial admin user
node src/scripts/create_default_admin.js
# Start backend dev server (http://localhost:5000)
npm run dev
```

3. Frontend Setup (in a new terminal)

```
cd ../frontend
npm install
# Start frontend dev server (http://localhost:5173)
npm run dev
```

Verifying the Setup

- Open <http://localhost:5173> — public homepage should load
- Navigate to <http://localhost:5173/team/login>
- Log in with the credentials set in `create_default_admin.js`
- Backend health check: GET <http://localhost:5000/health>

ngrok Development (for remote access/mobile testing)

The Vite config already allows `.ngrok-free.dev` subdomains. Run:

```
ngrok http 5173
```

The public URL will work immediately without config changes.

15. Docker / Production Deployment

A `docker-compose.yml` is provided at the root:

```
# From lrp-pune_main/ directory
docker compose up --build
```

This starts 4 services:

Service	Container	Port	Description
PostgreSQL 15	lrp_postgres	5432	Database
Redis 7	lrp_redis	6379	Queue (for BullMQ)
Backend	lrp_backend	5000	Express API
Frontend	lrp_frontend	80	Nginx-served React build

⚠ CRITICAL The `docker-compose.yml` contains plaintext default secrets (`password123`, `super_secure_enterprise_secret_key`). These must be changed before any production deployment.

GitHub Actions CI

A CI workflow exists at `.github/workflows/ci.yml`. Review this file to understand what automated checks run on push.

16. Testing

```
cd backend
npm test # or: npm.cmd test (Windows)
```

Test suites are in `backend/src/tests/`:

Test File	Coverage
<code>auth.test.js</code>	Login, registration, session
<code>dataEntry.test.js</code>	Family field edits
<code>searchAndFamily.test.js</code>	Search, family retrieval
<code>workbookGeneration.test.js</code>	Workbook generation pipeline
<code>workbookImport.test.js</code>	Excel import pipeline
<code>adminWorkbook.test.js</code>	Admin workbook management

⚠ CRITICAL Tests use a live database connection. Ensure `DATABASE_URL` points to a test database before running.

17. Known Issues, Technical Debt & Gotchas

🔍 Critical Issues

1. Hardcoded Template Path

`generatorService.js` line 11 contains an absolute Windows path:

```
const TEMPLATE_PATH = "c:/Users/--/Desktop/LRP-PUNE/Revision 29.xlsx";
```

This only works on the original developer's machine. The fallback to `activeWorkbook.filePath` works if a workbook was properly imported. Fix: Remove the hardcoded path and require an active workbook to always be set.

2. Live Credentials in `.env`

The `backend/.env` file contains live production credentials (Neon DB, Gemini API key, JWT secret). These must be rotated.

3. CORS Policy

`app.js` uses `origin: true` (allow all origins). This should be locked to specific allowed origins in production.

4. `/auth/register` is Unprotected

Anyone can create admin accounts without authentication. This endpoint should be removed or admin-gated in production.

-> Technical Debt

- Duplicate master-workbook / master-data directories in the frontend. Both `src/master-data/` and `src/master-workbook/` exist with some duplicate components (`WorkbookHistory.jsx`, `GenerationProgressDialog.jsx`).
- BullMQ/Redis installed but not wired — `bullmq` and `ioredis` are in `package.json` but the queue system is not actively used in any business logic.
- `SOURCE_CODE_STRUCTURE.md` references old file paths from a previous developer machine (`/Users/dellb/Desktop/LRP-PUNE-BF/`) — paths are broken/stale.
- Static chart data in `dashboardService.js` — The `programGrowth` chart data is static/hardcoded. It does not come from the database.
- `validationEngine.js` and `validationRunner.js` — The validation system is wired but the actual business rules within `validationEngine.js` should be reviewed to ensure they match the current workbook column structure.
- `PillarPage` components have large inline config objects — Each `PillarPage-N.jsx` has a massive static config object with fallback values. These could be extracted to a shared constants file.

-> Notes / Gotchas

- Pillar Page Data Loading — Pillar pages first render with static fallback values, then overlay live data from `usePillar(N)`. If the API is slow, users see static content briefly.
- Metric Commit is One-Way — Once metrics from a document are `COMMITTED` or `EXCLUDED`, they cannot be re-opened. You must re-upload the document to re-extract.
- Family Deletion via Import — Every workbook import via `dbImporter.js` deletes all existing `Family`, `FamilyField`, and `WorkbookHeader` data first. There is no incremental import — it's always a full replacement.
- Column A and E are Immutable — `hhCode` (col A) and `ownerName` (col E) cannot be edited via the UI. `isEditable = false` is enforced in the importer for these columns.
- ngrok URLs in Vite Config — The specific ngrok hostnames in `vite.config.js` are from the original developer's sessions and will be different for a new developer. The wildcard `.ngrok-free.dev` entry should cover new tunnels.

18. Key Files Quick Reference

File	Purpose
<code>backend/prisma/schema.prisma</code>	Full database schema
<code>backend/server.js</code>	Server entry point
<code>backend/src/app.js</code>	Express middleware & route mounting
<code>backend/src/config/permissions.js</code>	RBAC role-permission matrix
<code>backend/src/config/minio.js</code>	MinIO client configuration
<code>backend/src/services/aiService.js</code>	Gemini AI metrics extraction
<code>backend/src/services/documentService.js</code>	Full document ingestion pipeline
<code>backend/src/services/authService.js</code>	Auth business logic
<code>backend/src/master-workbook/importer/dbImporter.js</code>	Excel workbook → PostgreSQL

File	Purpose
backend/src/master-workbook/generator/generatorService.js	PostgreSQL → Excel generation
backend/src/routes/adminRoutes.js	All admin API routes
backend/src/routes/documentRoutes.js	Document hub routes
backend/.env	All secrets (rotate immediately)
frontend/src/routes/AppRoutes.jsx	All frontend route definitions
frontend/src/store/authStore.js	Zustand auth state
frontend/src/service/apiClient.js	Axios instance with 401 interceptor
frontend/vite.config.js	Vite dev server proxy config
docker-compose.yml	Container orchestration
README.md	AI agent onboarding guide

Document generated from full source code analysis on 27 July 2026.